

Full Length Article

Distilling mathematical reasoning capabilities into Small Language Models

Xunyu Zhu ^{a,b}, Jian Li ^{a,b,*}, Yong Liu ^c, Can Ma ^{a,b}, Weiping Wang ^{a,b}^a Institute of Information Engineering, Chinese Academy of Sciences, China^b School of Cyber Security, University of Chinese Academy of Sciences, China^c Gaoling School of Artificial Intelligence, Renmin University of China, China

ARTICLE INFO

Keywords:

Large language models
Knowledge Distillation
Mathematical reasoning
Chain-of-Thought
Program-of-Thought

ABSTRACT

This work addresses the challenge of democratizing advanced Large Language Models (LLMs) by compressing their mathematical reasoning capabilities into sub-billion parameter Small Language Models (SLMs) without compromising performance. We introduce Equation-of-Thought Distillation (EoTD), a novel technique that encapsulates the reasoning process into equation-based representations to construct an EoTD dataset for fine-tuning SLMs. Additionally, we propose the Ensemble Thoughts Distillation (ETD) framework to enhance the reasoning performance of SLMs. This involves creating a reasoning dataset with multiple thought processes, including Chain-of-Thought (CoT), Program-of-Thought (PoT), and Equation-of-Thought (EoT), and using it for fine-tuning. Our experimental performance demonstrates that EoTD significantly boosts the reasoning abilities of SLMs, while ETD enables these models to achieve state-of-the-art reasoning performance.

1. Introduction

Large language models (LLMs) like those built on Transformer architectures mark a leap forward in natural language processing. These models, including prominent ones such as LLaMA (Touvron, Lavril et al., 2023), GPT-4 (OpenAI, 2023), and PaLM (Chowdhery et al., 2023), boast parameter counts in the hundreds of billions. Trained on vast text datasets, they demonstrate remarkable proficiency in a wide array of downstream tasks.

Recent studies (Chen, Ma, Wang, & Cohen, 2023; Liu et al., 2023; Wang, Wei et al., 2023; Wang, Xu et al., 2023) have honed the reasoning abilities of LLMs through chain-of-thought (CoT) prompting, which generates intermediate steps to solve complex problems. However, the deployment of such models is challenging due to their size and computational requirements. For example, the GPT-3 model (Brown et al., 2020a) necessitates at least 320 GB of FP16 storage and multiple A100 GPUs with 80 GB of memory each for efficient inference.

Recent work (Fu, Peng, Ou, Sabharwal, & Khot, 2023; Ho, Schmid, & Yun, 2023; Magister, Mallinson, Adamek, Malmi, & Severyn, 2023; Shridhar, Stolfo, & Sachan, 2023) investigates distilling LLM reasoning into SLMs (under 1B parameters) for broader deployment. This involves using LLMs to create enriched datasets with detailed reasoning paths, which then fine-tune SLMs, endowing them with advanced reasoning abilities. For example, Chain-of-Thought Distillation (CoTD) (Ho et al., 2023) encapsulates the reasoning process into textual rationales, and Program-of-Thought Distillation (PoTD) (Zhu, Qi, Zhang, Long &

Zhou, 2023) formulates the reasoning process into python program. The methodologies for CoTD and PoTD are delineated in Appendix A and Appendix B, respectively. These distillation methods have distinct strengths and limitations. As illustrated in Fig. 1, CoTD instructs SLMs to generate a step-by-step reasoning flow in natural language and perform calculations in real-time, offering a flexible solution format but risking incorrect answers due to calculation errors (Ho et al., 2023). PoTD addresses this issue by training SLMs to conceptualize questions as programs, which are then executed by a Python interpreter to produce the final answer. However, SLMs under PoTD sometimes generate unknown variables, but the python interpreter can execute programs only if every variable of programs is defined with a value. Another intriguing method involves framing math problems as linear equation systems (Liu et al., 2023). Inspired by this, we introduce Equation-of-Thought Distillation (EoTD), teaching SLMs to model math questions in a similar manner. EoTD facilitates a direct understanding of mathematical principles and fosters logical thinking in SLMs.

The diversity within each distillation method does not signify rivalry or exclusivity. Instead, in practical problem-solving situations, employing multiple methods can offer complementary advantages. These distinct distillation approaches can synergize, resulting in benefits that exceed those achievable with any single approach. Motivated by this, we introduce Ensemble Thoughts Distillation (ETD) to further enhance SLM reasoning. ETD merges the CoTD, PoTD, and EoTD datasets into

* Corresponding author at: Institute of Information Engineering, Chinese Academy of Sciences, China.

E-mail addresses: zhuxunyu@iie.ac.cn (X. Zhu), lijian9026@iie.ac.cn (J. Li), liuyonggsai@ruc.edu.cn (Y. Liu), macan@iie.ac.cn (C. Ma), wangweiping@iie.ac.cn (W. Wang).

<https://doi.org/10.1016/j.neunet.2024.106594>

Received 27 March 2024; Received in revised form 17 June 2024; Accepted 31 July 2024

Available online 2 August 2024

0893-6080/© 2024 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

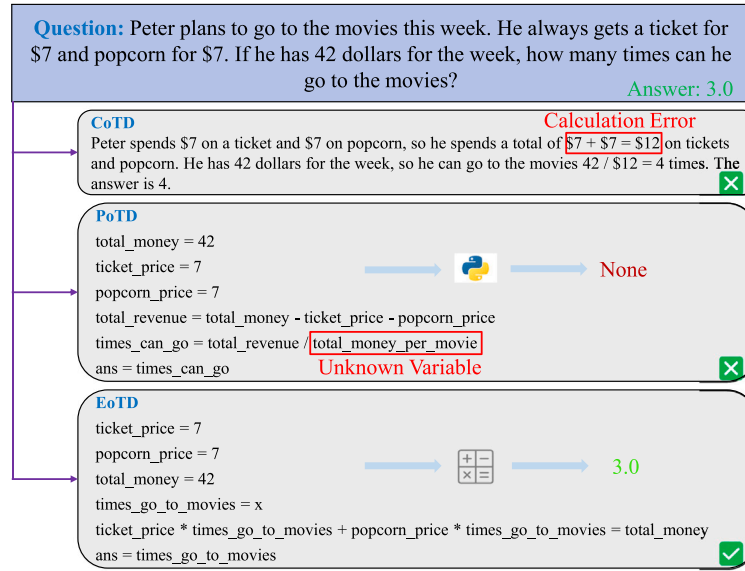


Fig. 1. A particular case where SLMs under CoTD and PoTD fail to generate the correct answer, but SLMs under EoTD successfully solve the question.

a comprehensive ETD dataset for fine-tuning SLMs. The diversity of reasoning strategies within the ETD dataset enriches the reasoning knowledge, contributing to its effectiveness.

We assessed EoTD and ETD across CodeT5 models from Small (0.06B) to Large (0.77B) on four mathematical reasoning datasets. Results indicate EoTD significantly boosts SLM reasoning abilities, while ETD enables SLMs to reach state-of-the-art (SOTA) performance. For instance, with EoTD, CodeT5-Small reached 18.87% accuracy on GSM8K, and ETD elevated CodeT5-Large to 42.45% accuracy. Ablation studies confirm that the volume and variety of reasoning paths in ETD correlate with improved SLM reasoning performance.

2. Related work

2.1. Large language models (LLMs)

Building on the insights from Huang and Chang (2023), Wei, Bosma et al. (2022), Wei, Wang et al. (2022), our research investigates the distillation of complex reasoning pathways from LLMs, such as GPT-4 (OpenAI, 2023) and PaLM-2 (Anil et al., 2023), into more manageable models. These LLMs, with their vast parameter counts exceeding 100 billion, have demonstrated a profound capacity for navigating intricate reasoning tasks. They can independently construct a series of logical steps leading to a conclusion, particularly when provided with structured reasoning examples or when guided by prompts that encourage stepwise thinking. Our work aims to capture this advanced reasoning in smaller models, thus reducing the computational overhead and making such capabilities more widely accessible.

The formidable reasoning skills of LLMs on complex tasks are offset by their extensive size and computational demands. Deploying models like GPT-3 (Brown et al., 2020b) for inference, for example, demands at least 320 GB of storage for FP16 parameters and no fewer than five A100 GPUs with 80 GB of memory each for efficient functioning. These requirements pose significant challenges, particularly for resource-limited settings. Our research addresses these issues by distilling the reasoning capabilities of LLMs into smaller, more computationally feasible models.

Our work addresses these limitations by focusing on the distillation of reasoning abilities from LLMs into smaller models. This process aims to retain the advanced reasoning capabilities of LLMs while significantly reducing resource requirements. Consequently, our approach facilitates the democratization of cutting-edge NLP technologies, enabling the use of powerful reasoning tools in settings with constrained computational resources.

2.2. Mathematical reasoning

Mathematical Reasoning tasks, highlighted by benchmarks like GSM8K (Cobbe et al., 2021) and SVAMP (Patel, Bhattamishra, & Goyal, 2021), pose a significant challenge for LLMs. To improve LLMs' performance in this domain, researchers have pinpointed two main strategies.

Chain-of-Thought Reasoning LLMs' reasoning can be enhanced by prompting them to articulate intermediate steps towards a solution, as demonstrated by Wei, Wang et al. (2022). This insight has led to various advancements (Chen et al., 2023; Liu et al., 2023; Wang, Wei et al., 2023; Wang, Xu et al., 2023) that refine reasoning paths. For example, Chen et al. (2023) prompt LLMs to generate executable code, Wang, Wei et al. (2023) use multiple reasoning paths with a voting mechanism for the correct answer, Wang, Xu et al. (2023) have LLMs create a plan before reasoning, and Liu et al. (2023) employ diverse reasoning prompts for problem-solving. Building on these methods, our work introduces Equation-of-Thought Distillation (EoTD) to further improve SLMs' mathematical reasoning.

Finetuning-based Reasoning refines LLMs like Llama2 (Touvron, Martin et al., 2023), Qwen (Bai et al., 2023), and Baichuan2 (Yang et al., 2023) by drawing on techniques from advanced models such as GPT-4 (OpenAI, 2023) and PaLM-2 (Anil et al., 2023). Notably, Yuan et al. (2023) employ Rejection Sampling Fine-Tuning (RFT) to enhance LLMs' mathematical reasoning, while WizardMath (Luo et al., 2023) uses Reinforcement Learning from Evolved Instructions Feedback (RLEIF) to improve LLaMA-2's reasoning abilities. MAMMO TH (Yue et al., 2023) combines CoT and PoT rationales for more effective instruction-tuning of LLMs in math problem-solving. Despite their effectiveness, the large size of these LLMs limits their deployment efficiency.

2.3. Knowledge distillation

Knowledge Distillation optimizes LLMs for practical use by transferring knowledge from larger models to smaller, efficient ones (Zhu, Li, Liu, Ma & Wang, 2023). Research (Fu et al., 2023; Ho et al., 2023; Magister et al., 2023; Shridhar et al., 2023) has aimed to endow compact models like T5 (Raffel et al., 2020) and GPT-2 (Radford et al., 2019) with the advanced reasoning of LLMs such as GPT-4 (OpenAI, 2023) and PaLM-2 (Anil et al., 2023). For example, Ho et al. (2023) fine-tune student models using the most accurate reasoning paths from LLMs. Shridhar et al. (2023) train a dual-model system on sub-questions

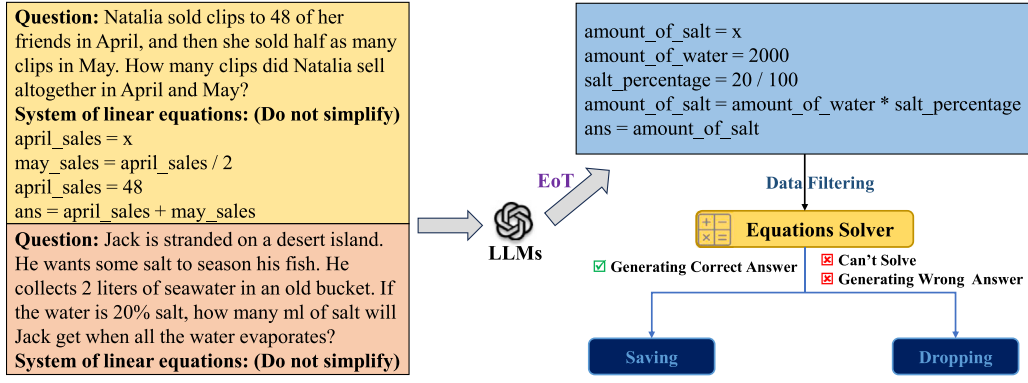


Fig. 2. Detailed data generation of our framework. Firstly, we manually construct some contextualized examples, and combine these contextualized examples, the question, and the prompt “System of linear equations: (Do not simplify)” to prompt LLMs to generate EoT based on the question. This equations system is sent to a deterministic equation solver, if there are compile errors or if it produces wrong answer, we will drop the EoT. Finally, we get a high-quality reasoning dataset.

and solutions, while Fu et al. (2023) suggest scaling down general competencies of smaller models to boost task-specific performance. Our work presents a novel distillation approach that encodes mathematical reasoning as equations and introduces Ensemble Thoughts Distillation, combining CoT, EoT, and PoT to create a diverse dataset with more abundant reasoning knowledge. Our results demonstrate state-of-the-art performance in mathematical reasoning.

3. Methodology

In this work, we introduce a novel distillation method for mathematical reasoning tasks, termed Equation-of-Thought Distillation (EoTD), which translates mathematical reasoning into equations for fine-tuning SLMs.

3.1. Equation-of-thought distillation

3.1.1. Data generation from LLMs

Our EoTD framework commences by creating a dataset from LLMs, which precedes SLM fine-tuning. As illustrated in Fig. 2, we employ in-context learning (Dong et al., 2023; Min et al., 2022; Rubin, Herzig, & Berant, 2022) to prompt LLMs for reasoning data. Within a mathematical dataset D , each entry (x, y) pairs a question x with its answer y . We select k samples $\{(x_1, y_1), \dots, (x_k, y_k)\}$ from D and manually craft rationales e in EoT format. These form contextualized instances $\{(x_1, e_1, y_1), \dots, (x_k, e_k, y_k)\}$, compiled into a demonstration set D_D . We then prompt LLMs with a question and the instruction “System of linear equations: (Do not simplify)” to generate rationales. The EoT generation is formalized as:

$$e_i = f_{\mathcal{M}}(x_i, D_D),$$

where $\mathcal{M}$ denotes the LLM, f the decoding function, and i the index in D . This yields the EoT dataset D_E , composed of triplets (x, e, y) .

Data Filtering—After LLMs generate EoT, we validate each equation system using an external Equation Solver to ensure the accuracy of our initial dataset D_E . Any equation system that cannot be solved or produces an incorrect result is excluded. This rigorous filtering process removes errors, enhancing the dataset’s quality. Consequently, this refinement directly improves the performance of fine-tuned SLMs by providing cleaner and more reliable training data.

3.1.2. Fine-tuning SLMs

After assembling the reasoning dataset D_E , we fine-tune SLMs on it. For each training instance (x, e, y) from D_E , we prepend the prompt p_e “System of linear equations: (Do not simplify)” to the question x

and employ instruction tuning to guide the SLM in generating the corresponding equations. The fine-tuning loss function is:

$$\mathcal{L} = - \sum_{i=1}^N \sum_{t=1}^T \log P(e_t^i | e_{<t}^i, x^i, p_e),$$

where N is the number of examples in D_E , p_e is the prompt, and $e_{:T}$ is the sequence of equations. Post fine-tuning, the SLM can generate equations for complex questions, which are then solved by an external Equation Solver to obtain the final answer.

3.2. Ensemble thoughts distillation

ETD merges the CoTD, PoTD, and EoTD to improve diversity of reasoning forms. The diverse reasoning forms have a richer knowledge of reasoning, and can be effective to improve reasoning ability of SLMs. We now detail the ETD method and its implementation.

In parallel with EoTD, we construct a Program-of-Thought (PoT) dataset D_p , with each entry formatted as a triplet (x, p, y) . The PoT data generation mirrors the EoT process described in Section 3.1.1. We utilize in-context learning to prompt LLMs to generate programmatic solutions for given questions. These programs are then executed by an external Python interpreter to obtain the final answers. Programs that fail to compile or yield incorrect answers are discarded, ensuring the PoT dataset D_p is of high quality. Similarly, we compile a Chain-of-Thought (CoT) dataset D_c , with each instance also structured as a triplet (x, c, y) . The methodologies for CoTD and PoTD are delineated in Appendix A and Appendix B, respectively.

As depicted in Fig. 3, we amalgamate the EoT, CoT, and PoT datasets to form the new ETD dataset D_{ETD} . For EoT entries, we append the prompt p_e “System of linear equations: (Do not simplify)” to each question. For PoT entries, we add the prompt p_p “Let’s break down the code step by step”, and for CoT entries, we include the prompt p_c “Let’s think step by step”. These prompts are designed to guide the generation of thoughts in their respective formats. The combined datasets, now enriched with diverse thoughts and instructions, constitute the ETD dataset. We then apply instruction tuning to fine-tune SLMs on D_{ETD} . The loss function for this fine-tuning process is:

$$\mathcal{L} = - \sum_{i=1}^N \sum_{t=1}^T \log P(r_t^i | r_{<t}^i, x^i),$$

where x is the input comprising both the question and its associated prompt, and r represents the generated thoughts conditioned on the input. This approach aims to enhance the SLMs’ ability to process and generate a variety of thought patterns, thereby improving their mathematical reasoning performance.

After fine-tuning, the SLMs are primed to generate different types of reasoning outputs based on the given prompts. When presented with

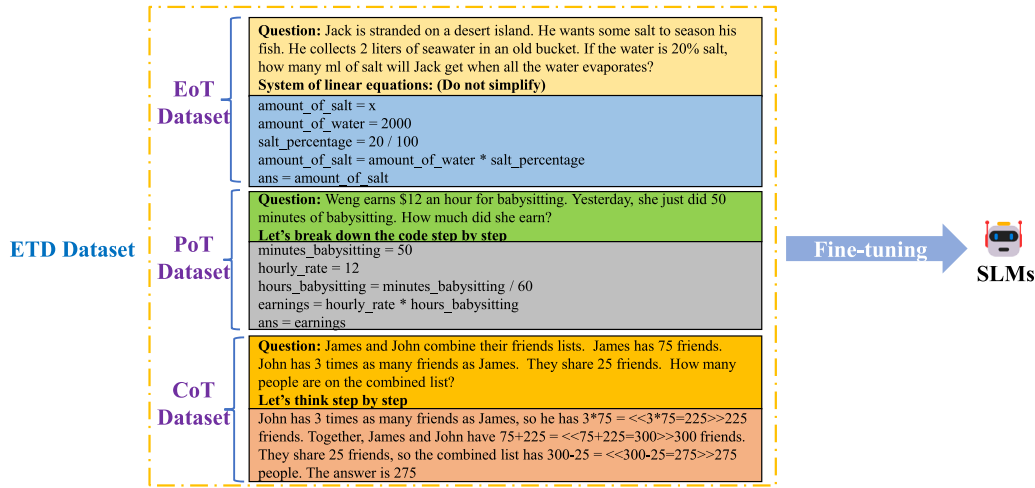


Fig. 3. Detailed overview of Ensemble Thought Distillation. Firstly, we combine a CoT dataset, a PoT dataset and a EoT dataset to build a new ETD dataset. The ETD dataset has diverse thoughts and prompts. Then, we use the ETD dataset to fine-tune SLMs. After fine-tuning, we use the prompt “System of linear equations: (Do not simplify)” to instruct SLMs to generate equations, the prompt “Let’s break down the code step by step” to instruct SLMs to generate programs, and the prompt “Let’s think step by step” to instruct SLMs to generate chains to solve questions.

Table 1
Size of the datasets used in our experiments. Drop Rate refers to the size of dropped data divided by the size of the sampled data.

Method	Sampled dataset	Filtered dataset	Drop Rate(%)
CoTD	29 892	24 392	18.4
PoTD	29 892	22 491	24.8
EoTD	29 892	15 946	46.7

a question, the prompt “System of linear equations: (Do not simplify)” elicits equation generation, “Let’s break down the code step by step” induces program generation, and “Let’s think step by step” prompts the creation of chains of thought. The SLMs’ outputs are then used to derive the final answers. As evidenced in Section 4.5, PoT outperforms the others in reasoning, with EoT in second place, and CoT trailing. Consequently, the PoT-generated answer is initially considered as the final answer. If the PoT-generated program fails to compile correctly, the SLMs’ EoT-generated answer is then evaluated. Should the EoT-generated equations be unsolvable, the CoT-generated result is finally considered. This hierarchical approach to reasoning ensures the most reliable answer is selected. The detailed reasoning process of ETD is illustrated in Fig. B.9.

4. Experiments

4.1. Dataset

Our training dataset is derived from the GSM8K (Cobbe et al., 2021) training set. We construct separate EoTD, PoTD, and CoTD datasets, which are then amalgamated to form the ETD dataset. This ensemble dataset features a variety of prompts and thought processes, on which we fine-tune SLMs. The mathematical reasoning capabilities of the SLMs are evaluated using the GSM8K (Cobbe et al., 2021) test set, as well as ASDiv (Miao, Liang, & Su, 2020), SVAMP (Patel et al., 2021), and MultiArith (Roy & Roth, 2015).

4.2. Implementation

We employ ChatGPT (gpt-3.5-turbo) as the teacher LLM to construct our training dataset and utilize CodeT5 models — Small (60M), Base (220M), and Large (770M) (Wang, Wang, Joty, & Hoi, 2021) — as student SLMs. We manually create 8 examples to guide ChatGPT in generating 4 reasoning paths for each dataset (EoT, PoT, and CoT),

and Table 1 shows the size of the datasets used in our experiments. Fine-tuning of all student SLMs is conducted using the Huggingface library (Wolf et al., 2020) on an NVIDIA 3090 GPU with 24 GB RAM. The learning rate for fine-tuning is set to $5e-4$, with a total of 10 fine-tuning epochs.

4.3. Baselines

Proprietary Large Language Models We present CoT prompting results from an array of SoTA LLMs, such as OpenAI’s GPT-4, ChatGPT (gpt-3.5-turbo), Google’s PaLM-2, and Anthropic’s Claude-2.

Open-Source Large Language Models We present mathematical reasoning performance of Llama-2-7B, CodeLLaMA-7B, and their fine-tuned versions, such as Platpus-2, WizardMath, TORA.

Fine-tuned Small Language Models We present some works that try to fine-tune SLMs under 1B, such as Ho et al. (2023) fine-tune GPT-3-ada, Fu et al. (2023) fine-tune FlanT5, and Shridhar et al. (2023) fine-tune GPT-2.

4.4. Main results

Table 2 showcases our method’s performance on four mathematical datasets, revealing key insights: (1) EoTD significantly enhances the mathematical reasoning of SLMs, with absolute improvements ranging from 25.51% to 43.52% across tasks. The key difference between EoTD and baseline approaches lies in the form of reasoning. Baselines typically rely on CoTD, which involves generating numerous steps and performing extensive calculations. While CoT has been shown to significantly improve the mathematical reasoning of LLMs, it is challenging for SLMs due to their limited capacity. In contrast, EoTD delegates the computational tasks to an Equation Solver, allowing the model to focus solely on generating reasoning steps. This approach significantly reduces the cognitive load on the model. Moreover, EoTD’s generation of equations provides a more intuitive representation of the relationships between variables, aiding SLMs in understanding and analyzing problems. Therefore, compared to CoTD, EoTD is better suited to improving the mathematical reasoning ability of SLMs. (2) ETD outperforms previous state-of-the-art fine-tuned SLMs at all scales, with absolute improvements between 48.14% and 57.58% across tasks. Furthermore, ETD’s accuracy is approximately 20% higher than that of EoTD, underscoring the advantage of diverse prompts and thoughts in bolstering SLMs’ reasoning capabilities. Earlier distillation datasets

Table 2

Overall test set performance. We use EoTD and ETD to fine-tune SLMs, and evaluate them on four mathematical reasoning datasets, i.e., GSM8K, ASDiv, SVAMP, and MultiArith. The experiment results show that EoTD can effectively improve SLMs' reasoning performance, and ETD makes SLMs achieve SOTA reasoning performance.

Models	#Params	GSM8K	ASDiv	SVAMP	MultiArith	AVG
<i>Proprietary Large Language Models</i>						
GPT-4 (OpenAI, 2023)	–	92.0	91.3	93.1	–	92.13
ChatGPT	–	80.8	87.3	83.0	–	83.7
Claude-2 (Anthropic, 2023)	–	85.2	–	–	–	85.2
PaLM-2 (Anil et al., 2023)	540B	80.7	–	–	–	80.7
<i>Open-Source Large Language Models</i>						
Llama-2 (Touvron, Martin et al., 2023)	7B	13.3	50.7	38.0	–	34
CodeLlama (Rozière et al., 2023)	7B	34.0	61.4	59.0	–	51.46
Platypus-2 (Lee, Hunter, & Ruiz, 2023)	7B	14.4	47.9	36.7	–	33
WizardMath (Luo et al., 2023)	7B	54.9	59.1	57.3	–	57.1
TORA (Gou et al., 2023)	7B	68.8	73.9	68.2	–	70.3
<i>Fine-tuned Small Language Models</i>						
Ho et al. (2023)	0.3B	3.11	–	–	–	3.11
Fu et al. (2023)	0.76B	20.2	23.8	20.4	38.5	25.72
Fu et al. (2023)	0.25B	13.4	20.9	14.2	29.7	19.55
Shridhar et al. (2023)	0.77B	17.89	–	18.14	–	18.01
Zhu, Qi et al. (2023)	0.77B	39.2	51.2	48.2	79.2	54.45
<i>Our fine-tuned Small Language Models</i>						
CodeT5-Small	0.06B	1.1	0.3	0.2	0.6	0.55
(+) EoTD		18.87	29.24	31.5	24.66	26.06
(+) ETD		33.58	49.09	42.8	67.83	48.14
CodeT5-Base	0.22B	0.8	0.2	0.0	0.0	0.25
(+) EoTD		27.21	38.26	38.8	41.66	36.48
(+) ETD		40.63	51.66	48.8	81	55.52
CodeT5-Large	0.77B	2.9	3.6	0.0	0.0	1.62
(+) EoTD		33.13	44.03	46.1	57.33	45.14
(+) ETD		42.45	52.81	49.59	85.5	57.58

predominantly used a single form of reasoning, limiting SLMs to learning fragmented reasoning knowledge. However, different reasoning forms have distinct focal points: CoT offers clear intermediate steps, making reasoning processes easier to understand and interpret, thus increasing transparency and credibility. EoT, based on mathematical principles and formulas, ensures high rigor and accuracy. PoT allows for reasoning automation via programming, boosting reasoning efficiency and accuracy. By generating datasets with multiple reasoning forms, SLMs can learn mathematical reasoning from various perspectives, thereby improving their overall mathematical reasoning capabilities. (3) Model size is crucial for reasoning distillation efficacy in SLMs; larger models assimilate more reasoning knowledge, translating to superior performance. For instance, under ETD, CodeT5-Small attains 33.58% accuracy on GSM8K, CodeT5-Base reaches 40.63%, and CodeT5-Large achieves 42.45%.

4.5. ETD enhances thoughts distillation

In this subsection, we examine if ETD can enhance the reasoning abilities of SLMs across different thought processes. Initially, we generate CoTD, PoTD, and EoTD datasets from the GSM8K training set, each containing one reasoning path per question. These datasets are then merged to create the ETD dataset. Subsequently, we fine-tune SLMs, including CodeT5-Small, Base, and Large, using the ETD dataset. The reasoning performance of these SLMs is assessed on the GSM8K test dataset, as well as ASDiv, SVAMP, and MultiArith.

Fig. 4 illustrates the outcomes of our experiments, from which we deduce that: (1) ETD enhances SLMs' reasoning performance. SLMs fine-tuned with ETD outperform those trained on CoTD, PoTD, and EoTD in CoT, PoT, and EoT tasks, respectively. For instance, CodeT5-Base achieves a 26.38% PoT accuracy on the GSM8K test dataset under ETD, surpassing the 22.44% PoT accuracy under PoTD. Similarly, CodeT5-Small reaches a 24.09% EoT accuracy on SVAMP with ETD, compared to 17.59% with EoTD. (2) SLMs gain more valuable reasoning knowledge from PoTD, reflected in superior reasoning performance,

with EoTD and CoTD following in that order. This pattern persists with ETD, leading to a hierarchical approach where the PoT-generated answer is preferred, followed by EoT if PoT fails, and CoT as a last resort. (3) Scaling up the size of student models consistently improves the performance across CoTD, PoTD, EoTD, and ETD, indicating that larger models benefit more from our method.

4.6. The effect of different thoughts in ETD

In this subsection, we investigate the impact of different reasoning paths within the ETD framework. We begin by fine-tuning CodeT5-Base on individual datasets — CoTD dataset, PoTD dataset, and EoTD dataset — each containing a unique reasoning path per question. We then extend our fine-tuning to combinations of these datasets: CoTD with PoTD dataset, CoTD with EoTD dataset, PoTD with EoTD dataset, and the full ETD dataset which integrates CoTD, PoTD, and EoTD. This approach allows us to assess the influence of each reasoning path and their synergistic effects on the model's performance.

Table 3 presents the results of our experiments, from which we observe that: (1) SLMs exhibit improved reasoning performance with an increasing number of thought processes incorporated into ETD. For instance, CodeT5-Base, when trained on CoTD and PoTD combined, achieves a 25.01% PoT accuracy on the GSM8K test dataset, which further increases to 26.38% when trained on the full combination of CoTD, PoTD, and EoTD. (2) SLMs trained on the PoTD and EoTD combination outperform those trained on either the CoTD and PoTD or the CoTD and EoTD combinations. This suggests that the structured nature of PoTD and EoTD contributes to SLMs' ability to assimilate more valuable knowledge effectively.

4.7. More data improves reasoning ability of SLMs

In this subsection, we explore the influence of dataset size on the reasoning capabilities of SLMs. We create subsets of varying sizes from our reasoning datasets and utilize these to fine-tune CodeT5-Base. This

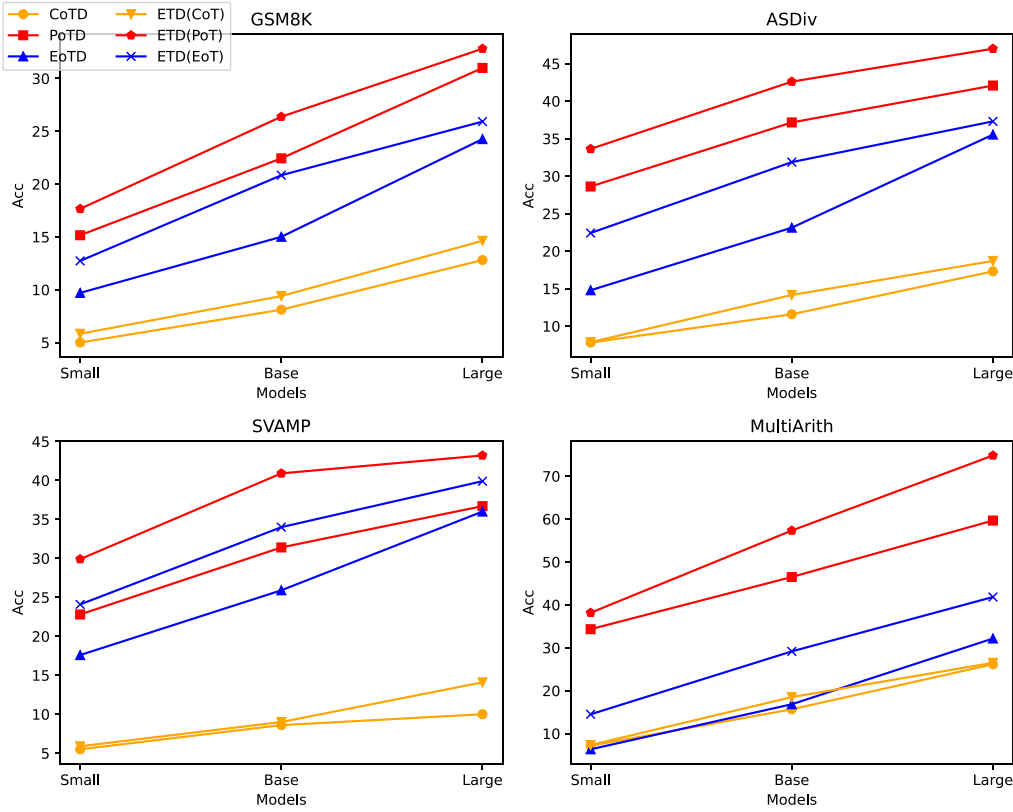


Fig. 4. Effect of ETD. We fine-tune SLMs on the ETD dataset, the CoTD dataset, the PoTD dataset, and the EoTD dataset to study the effect of ETD. The experiment results shows that ETD can improve reasoning performance of SLMs under different thoughts.

Table 3

Effect of Thoughts in ETD. We fine-tune CodeT5-Base on ETD datasets which own different thoughts in it to analyze the effect of thoughts in ETD. The experiment results show that when the ETD dataset own more thoughts, SLMs fine-tuned on it can achieve better reasoning performance.

Methods	GSM8K	ASDiv	SVAMP	MultiArith
CoTD	8.11	11.59	8.6	15.66
PoTD	22.44	37.16	31.4	46.5
EoTD	15.01	23.13	25.90	16.83
<i>CoTD + PoTD</i>				
CoT	7.96	13.12	11.3	14.16
PoT	25.01	39.59	35.6	53.16
<i>CoTD + EoTD</i>				
CoT	8.49	13.02	8.79	15.16
EoT	17.13	26.62	28.19	20.5
<i>PoTD + EoTD</i>				
PoT	26.23	39.59	34.8	59
EoT	18.65	29.10	31.5	25.5
<i>CoTD + PoTD + EoTD</i>				
CoT	9.4	14.16	9	18.5
PoT	26.38	42.6	40.9	57.33
EoT	20.84	31.87	34	29.16

analysis helps determine the relationship between the amount of data and the model's performance in reasoning tasks.

Fig. 5 depicts the outcomes of our experiments, indicating that larger datasets enhance the reasoning performance of SLMs. For instance, CodeT5-Base, when fine-tuned on a 1K ETD dataset, attains a 24.42% PoT accuracy on the ASDiv test set, whereas training on a smaller 0.5 K ETD dataset results in a lower PoT accuracy of 17.17% on the same test set. This trend underscores the positive correlation between dataset size and the model's reasoning proficiency.

4.8. Diverse reasoning paths improve SLMs' reasoning performance

In this subsection, we fine-tune CodeT5-Base on our reasoning datasets, which are differentiated by the number of reasoning paths they contain, to analyze the effect of reasoning path multiplicity on the reasoning performance of SLMs. This examination aims to discern how the diversity and quantity of reasoning paths in training data influence the model's ability to perform reasoning tasks.

Fig. 6 presents the results of our experiments, which demonstrate that a variety of reasoning paths can bolster the reasoning performance of SLMs. For instance, CodeT5-Base, when trained on an ETD dataset featuring four reasoning paths, attains a 38.89% PoT accuracy on the GSM8K test dataset and a 44.13% EoT accuracy on ASDiv. In contrast, CodeT5-Base trained on an ETD dataset with only one reasoning path achieves the same 38.89% PoT accuracy on GSM8K but a lower 31.87% EoT accuracy on ASDiv. This suggests that the inclusion of multiple reasoning paths in training data can significantly enhance the model's performance, particularly in tasks requiring explanation generation.

5. Conclusion

Our research improves how smaller language models (SLMs) think by introducing two new techniques, Equation-of-Thought Distillation (EoTD) and Ensemble Thoughts Distillation (ETD). These methods enable SLMs to perform complex mathematical reasoning. EoTD breaks down reasoning into simpler equations, while ETD uses a range of examples to boost performance. Our findings show that these approaches enhance the SLMs' reasoning skills, making them suitable for use in environments with limited computing resources. However, there are still some limitations hindering the further improvement of our method's mathematical reasoning capability. The first limitation is that LLMs are typically pretrained on natural language and program

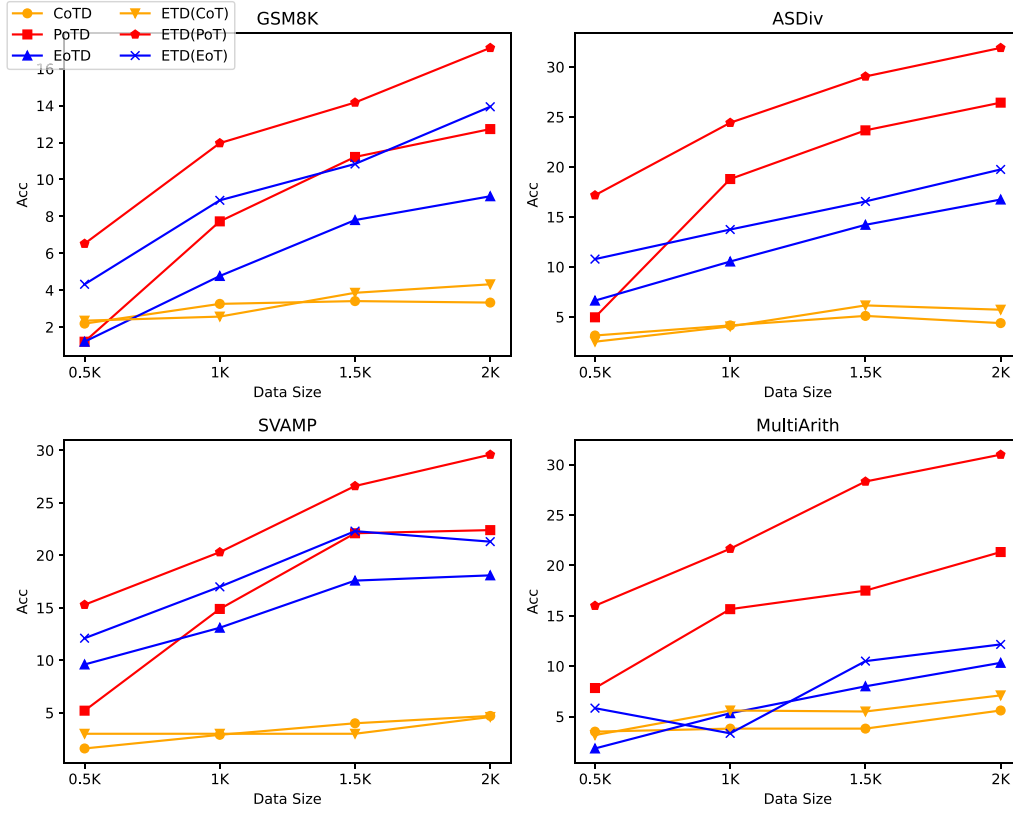


Fig. 5. Effect of Data Scale. We fine-tune CodeT5-Base under different data sizes to evaluate the effect of data scale. The experiment results show that larger data size make SLMs better reasoning performance.

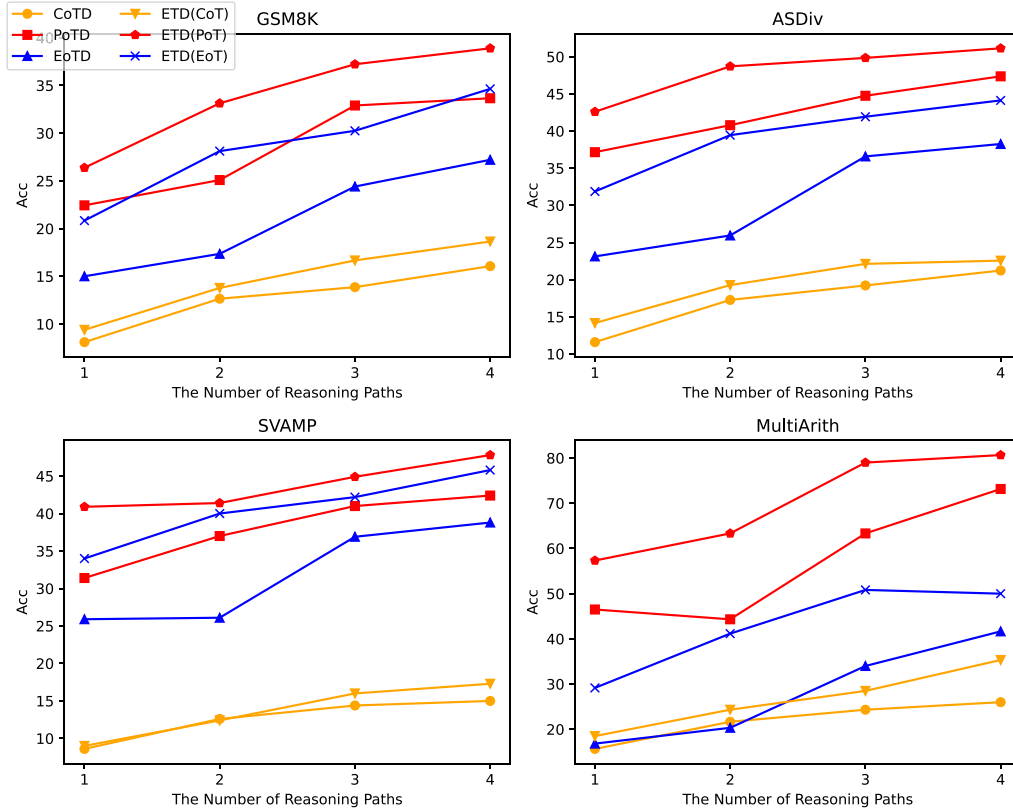


Fig. 6. Effect of Reasoning Paths. We fine-tune CodeT5-Base with different reasoning paths to analyze the effect of reasoning paths. The experiment results shows that diverse reasoning paths can improve SLMs' reasoning performance.

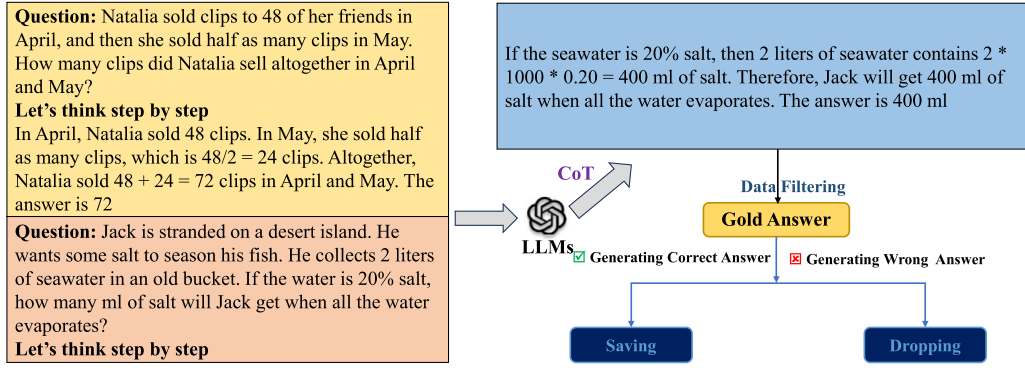


Fig. 7. Detailed data generation of CoTD. Firstly, we manually construct some contextualized examples, and combine these contextualized examples, the question, and the prompt “Let’s think step by step” to prompt LLMs to generate CoT based on the question. Then, We extract the answer from this rationale. If the answer does not agree with the gold answer, we will drop the CoT. Finally, we get a high-quality reasoning dataset.

data. Consequently, when using EoT, they exhibit poorer mathematical reasoning performance compared to CoT and PoT, resulting in a smaller EoTD dataset size. The second limitation is that SLMs are also pretrained on natural language and program data. Thus, fine-tuning SLMs with CoTD and PoTD datasets enhances their reasoning ability, whereas EoTD lacks corresponding SLMs, limiting its potential to improve SLMs’ mathematical reasoning performance. We are working on further overcoming these limitations and exploring our method’s application beyond mathematics to enhance SLM versatility.

CRedit authorship contribution statement

Xunyu Zhu: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Data curation. **Jian Li:** Supervision, Project administration, Funding acquisition. **Yong Liu:** Supervision. **Can Ma:** Supervision. **Weiping Wang:** Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

The work of Jian Li is supported partially by National Natural Science Foundation of China (No. 62106257). The work of Yong Liu is supported partially by National Natural Science Foundation of China (No. 62076234), Beijing Outstanding Young Scientist Program, China (No. BJJWZYJH012019100020098), the Unicom Innovation Ecological Cooperation Plan, China, and the CCF-Huawei Populus Grove Fund, China.

Appendix A. Chain-of-Thought distillation

A.1. Data generation from LLMs

The CoTD process commences with the generation of a dataset from LLMs, which lays the groundwork for subsequent fine-tuning of SLMs. As illustrated in Fig. 7, we employ in-context learning strategies (Dong et al., 2023; Min et al., 2022; Rubin et al., 2022) to elicit the generation of rationales from LLMs within a mathematical reasoning dataset D , where each entry is a tuple (x, y) — x being the

question and y the correct answer. To generate CoT, we select k samples $\{(x_1, y_1), (x_2, y_2), \dots, (x_k, y_k)\}$ from D and manually craft corresponding rationales c in CoT format. These form contextualized examples $\{(x_1, c_1, y_1), (x_2, c_2, y_2), \dots, (x_k, c_k, y_k)\}$, which are compiled into a demonstration set D_D . We then prompt the LLM with a new question appended with “Let’s think step by step” and feed it the demonstration set to generate a rationale for the question. The CoT generation formula is:

$$c_i = f_{\mathcal{M}}(x_i, D_D),$$

where $\mathcal{M}$ denotes the LLM, f is the greedy decoding function, and i indexes the example (x, y) in D . This procedure results in a CoT dataset D_P , composed of triplets (x, c, y) .

Data Filtering—Upon generating CoT with LLMs, we validate the rationale against the gold answer, a crucial step to ensure the quality of our initial reasoning dataset D_C . Discrepancies between the rationale’s answer and the gold answer result in exclusion from D_C . This filtering meticulously purges incorrect instances, enhancing the dataset’s quality. Consequently, this refinement directly contributes to the enhanced performance of fine-tuned SLMs, attributable to the increased accuracy and dependability of the training data.

A.2. Fine-tuning SLMs

After assembling the reasoning dataset D_C , we fine-tune SLMs on it. For each training instance (x, c, y) from D_C , we prepend the prompt p_c “Let’s think step by step” to the question x to form the input. Instruction tuning is then applied to the SLM to generate the corresponding rationale. The loss function for fine-tuning is:

$$\mathcal{L} = - \sum_{i=1}^N \sum_{t=1}^T \log P(c_t^i | c_{<t}^i, x^i, p_c),$$

where N is the number of examples in D_C , p_c is the prompt guiding the SLM to generate the rationale c , and $c_{<t}$ represents the sequence of rationale steps $\{c_1, c_2, \dots, c_T\}$.

After fine-tuning, the SLM becomes proficient at initiating a reasoning process for complex questions, generating the corresponding rationale. The final answer is then extracted from this rationale.

Appendix B. Program-of-thought distillation

B.1. Data generation from LLMs

The initial phase in our PoTD entails creating a dataset from LLMs, setting the stage for SLM fine-tuning. Fig. A.8 outlines this data generation process. We utilize in-context learning methods (Dong et al., 2023;

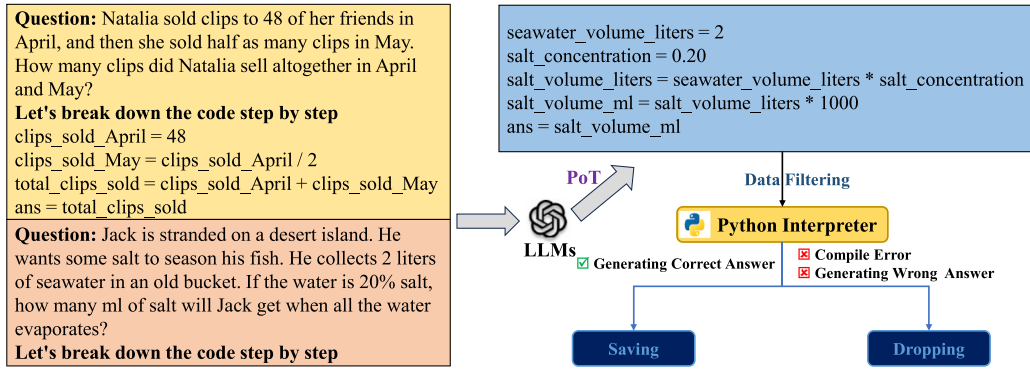


Fig. A.8. Detailed data generation of PoTD. Firstly, we manually construct some contextualized examples, and combine these contextualized examples, the question, and the prompt “Let’s break down the code step by step” to prompt LLMs to generate PoT based on the question. This program is sent to a extra python interpreter, if there are compile errors or if it produces wrong answer, we will drop the PoT. Finally, we get a high-quality reasoning dataset.

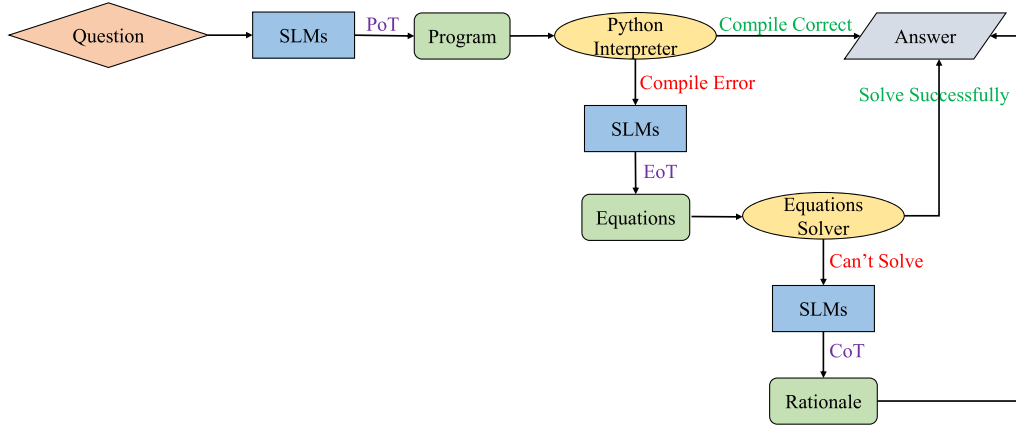


Fig. B.9. The detail reasoning process of ETD. When given a question, we first prompt SLMs to generate program in PoT, and send the program to an python interpreter to get the final answer. If the program fails to compile, we will prompt SLMs to generate equations in EoT, and send these equations to a equations solver to solve these equations to get the final answer. If these equations cannot be solved, we will prompt SLMs to generate rationale in CoT, and extract the final answer from rationale.

Min et al., 2022; Rubin et al., 2022) to induce LLMs to produce reasoning data. Within the mathematical reasoning dataset D , each entry is a tuple (x, y) , where x is the question and y the gold-standard answer. For PoT generation, we choose k samples $\{(x_1, y_1), (x_2, y_2), \dots, (x_k, y_k)\}$ from D and manually create rationales p in PoT format. These form contextualized instances $\{(x_1, p_1, y_1), (x_2, p_2, y_2), \dots, (x_k, p_k, y_k)\}$, which are compiled into a demonstration set D_D . We then prompt the LLM with a new question accompanied by “Let’s break down the problem step by step” and input the demonstration set to generate a rationale for the question. The PoT generation is formalized as:

$$p_i = f_{\mathcal{M}}(x_i, D_D),$$

where $\mathcal{M}$ is the LLM, f the greedy decoding function, and i the index of the instance (x, y) in D . This yields a PoT dataset D_p , organized as triplets (x, p, y) .

Data Filtering—Following PoT generation by LLMs, each program undergoes validation using an external Python interpreter, a vital step to ensure the quality of our initial dataset D_p . Programs that fail to compile or produce incorrect results are immediately discarded. This rigorous filtering process removes flawed instances, thus improving the dataset’s quality. The removal of these errors significantly enhances the performance of the fine-tuned SLMs due to the increased accuracy and dependability of the training data.

B.2. Fine-tuning SLMs

After generating the reasoning dataset D_p , we fine-tune SLMs on it. For each instance (x, p, y) from D_p , we append the prompt p_p “Let’s

break down the code step by step” to the question x for input, and apply instruction tuning to the SLM to produce the corresponding program. The fine-tuning loss function is:

$$\mathcal{L} = - \sum_{i=1}^N \sum_{t=1}^T \log P(p_t^i | p_{<t}^i, x^i, p_p),$$

where N is the count of examples in D_p , p_p is the prompt guiding the SLM to generate program p , and $p_{<T}$ represents the sequence of program steps $\{p_1, p_2, \dots, p_T\}$.

After fine-tuning, the SLM excels at initiating a reasoning process for complex questions, producing actionable programs. These are then executed by an external Python interpreter to obtain the final answer.

References

- Anil, R., Dai, A. M., Firat, O., Johnson, M., Lepikhin, D., Passos, A., et al. (2023). PaLM 2 technical report. <http://dx.doi.org/10.48550/ARXIV.2305.10403>, CoRR arXiv:2305.10403.
- Anthropic (2023). Model card and evaluations for claude models. *Anthropic Blog*, URL <https://www-files.anthropic.com/production/images/Model-Card-Claude-2.pdf?dm=1689034733>.
- Bai, J., Bai, S., Chu, Y., Cui, Z., Dang, K., Deng, X., et al. (2023). Qwen technical report. <http://dx.doi.org/10.48550/ARXIV.2309.16609>, CoRR arXiv:2309.16609.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., et al. (2020a). Language models are few-shot learners. In H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, H. Lin (Eds.), Vol. 33, *Advances in neural information processing systems* (pp. 1877–1901). Curran Associates, Inc., URL https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf.
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., et al. (2020b). Language models are few-shot learners. CoRR arXiv:2005.14165, URL <https://arxiv.org/abs/2005.14165>.

- Chen, W., Ma, X., Wang, X., & Cohen, W. W. (2023). Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research*, URL <https://openreview.net/forum?id=YfZ4Pt8zd>.
- Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., et al. (2023). PaLM: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24, 240:1–240:113, URL <http://jmlr.org/papers/v24/22-1144.html>.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., et al. (2021). Training verifiers to solve math word problems. CoRR arXiv:2110.14168, URL <https://arxiv.org/abs/2110.14168>.
- Dong, Q., Li, L., Dai, D., Zheng, C., Wu, Z., Chang, B., et al. (2023). A survey for in-context learning. <http://dx.doi.org/10.48550/ARXIV.2301.00234>, CoRR arXiv:2301.00234.
- Fu, Y., Peng, H., Ou, L., Sabharwal, A., & Khot, T. (2023). Specializing smaller language models towards multi-step reasoning. In A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, & J. Scarlett (Eds.), *Proceedings of machine learning research: Vol. 202, International conference on machine learning, ICML 2023, 23–29 July 2023, Honolulu, Hawaii, USA* (pp. 10421–10430). PMLR, URL <https://proceedings.mlr.press/v202/fu23d.html>.
- Gou, Z., Shao, Z., Gong, Y., Shen, Y., Yang, Y., Huang, M., et al. (2023). ToRA: A tool-integrated reasoning agent for mathematical problem solving. <http://dx.doi.org/10.48550/ARXIV.2309.17452>, CoRR arXiv:2309.17452.
- Ho, N., Schmid, L., & Yun, S.-Y. (2023). Large language models are reasoning teachers. In A. Rogers, J. Boyd-Graber, & N. Okazaki (Eds.), *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)* (pp. 14852–14882). Toronto, Canada: Association for Computational Linguistics, <http://dx.doi.org/10.18653/v1/2023.acl-long.830>, URL <https://aclanthology.org/2023.acl-long.830>.
- Huang, J., & Chang, K. C.-C. (2023). Towards reasoning in large language models: A survey. In A. Rogers, J. Boyd-Graber, & N. Okazaki (Eds.), *Findings of the association for computational linguistics: ACL 2023* (pp. 1049–1065). Toronto, Canada: Association for Computational Linguistics, <http://dx.doi.org/10.18653/v1/2023.findings-acl.67>, URL <https://aclanthology.org/2023.findings-acl.67>.
- Lee, A. N., Hunter, C. J., & Ruiz, N. (2023). Platypus: Quick, cheap, and powerful refinement of LLMs. <http://dx.doi.org/10.48550/ARXIV.2308.07317>, CoRR arXiv:2308.07317.
- Liu, T., Guo, Q., Yang, Y., Hu, X., Zhang, Y., Qiu, X., et al. (2023). Plan, verify and switch: Integrated reasoning with diverse X-of-thoughts. In H. Bouamor, J. Pino, K. Bali (Eds.), *Proceedings of the 2023 conference on empirical methods in natural language processing* (pp. 2807–2822). Singapore: Association for Computational Linguistics, <http://dx.doi.org/10.18653/v1/2023.emnlp-main.169>, URL <https://aclanthology.org/2023.emnlp-main.169>.
- Luo, H., Sun, Q., Xu, C., Zhao, P., Lou, J., Tao, C., et al. (2023). WizardMath: Empowering mathematical reasoning for large language models via reinforced evol-instruct. <http://dx.doi.org/10.48550/ARXIV.2308.09583>, CoRR arXiv:2308.09583.
- Magister, L. C., Mallinson, J., Adamek, J., Malmi, E., & Severyn, A. (2023). Teaching small language models to reason. In A. Rogers, J. Boyd-Graber, & N. Okazaki (Eds.), *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 2: short papers)* (pp. 1773–1781). Toronto, Canada: Association for Computational Linguistics, <http://dx.doi.org/10.18653/v1/2023.acl-short.151>, URL <https://aclanthology.org/2023.acl-short.151>.
- Miao, S.-y., Liang, C.-C., & Su, K.-Y. (2020). A diverse corpus for evaluating and developing english math word problem solvers. In D. Jurafsky, J. Chai, N. Schluter, & J. Tetreault (Eds.), *Proceedings of the 58th annual meeting of the association for computational linguistics* (pp. 975–984). Association for Computational Linguistics, <http://dx.doi.org/10.18653/v1/2020.acl-main.92>, Online, URL <https://aclanthology.org/2020.acl-main.92>.
- Min, S., Lyu, X., Holtzman, A., Artetxe, M., Lewis, M., Hajishirzi, H., et al. (2022). Rethinking the role of demonstrations: What makes in-context learning work? In Y. Goldberg, Z. Kozareva, Y. Zhang (Eds.), *Proceedings of the 2022 conference on empirical methods in natural language processing* (pp. 11048–11064). Abu Dhabi, United Arab Emirates: Association for Computational Linguistics, <http://dx.doi.org/10.18653/v1/2022.emnlp-main.759>, URL <https://aclanthology.org/2022.emnlp-main.759>.
- OpenAI (2023). GPT-4 technical report. <http://dx.doi.org/10.48550/ARXIV.2303.08774>, CoRR arXiv:2303.08774.
- Patel, A., Bhattamishra, S., & Goyal, N. (2021). Are NLP models really able to solve simple math word problems? In K. Toutanova, A. Rumshisky, L. Zettlemoyer, D. Hakkani-Tur, I. Beltagy, S. Bethard, R. Cotterell, T. Chakraborty, & Y. Zhou (Eds.), *Proceedings of the 2021 conference of the North American chapter of the association for computational linguistics: human language technologies* (pp. 2080–2094). Online: Association for Computational Linguistics, <http://dx.doi.org/10.18653/v1/2021.naacl-main.168>, URL <https://aclanthology.org/2021.naacl-main.168>.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al. (2019). Language models are unsupervised multitask learners. *OpenAI blog*, 1(8), 9, URL <https://insightcivic.s3.us-east-1.amazonaws.com/language-models.pdf>.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., et al. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21, 140:1–140:67, URL <http://jmlr.org/papers/v21/20-074.html>.
- Roy, S., & Roth, D. (2015). Solving general arithmetic word problems. In L. Márquez, C. Callison-Burch, & J. Su (Eds.), *Proceedings of the 2015 conference on empirical methods in natural language processing* (pp. 1743–1752). Lisbon, Portugal: Association for Computational Linguistics, <http://dx.doi.org/10.18653/v1/D15-1202>, URL <https://aclanthology.org/D15-1202>.
- Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., et al. (2023). Code llama: Open foundation models for code. <http://dx.doi.org/10.48550/ARXIV.2308.12950>, CoRR arXiv:2308.12950.
- Rubin, O., Herzig, J., & Berant, J. (2022). Learning to retrieve prompts for in-context learning. In M. Carpuat, M.-C. de Marneffe, & I. V. Meza Ruiz (Eds.), *Proceedings of the 2022 conference of the North American chapter of the association for computational linguistics: human language technologies* (pp. 2655–2671). Seattle, United States: Association for Computational Linguistics, <http://dx.doi.org/10.18653/v1/2022.naacl-main.191>, URL <https://aclanthology.org/2022.naacl-main.191>.
- Shridhar, K., Stolfo, A., & Sachan, M. (2023). Distilling reasoning capabilities into smaller language models. In A. Rogers, J. Boyd-Graber, & N. Okazaki (Eds.), *Findings of the association for computational linguistics: ACL 2023* (pp. 7059–7073). Toronto, Canada: Association for Computational Linguistics, <http://dx.doi.org/10.18653/v1/2023.findings-acl.441>, URL <https://aclanthology.org/2023.findings-acl.441>.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M., Lacroix, T., et al. (2023). LLaMA: Open and efficient foundation language models. <http://dx.doi.org/10.48550/ARXIV.2302.13971>, CoRR arXiv:2302.13971.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., et al. (2023). Llama 2: Open foundation and fine-tuned chat models. <http://dx.doi.org/10.48550/ARXIV.2307.09288>, CoRR arXiv:2307.09288.
- Wang, Y., Wang, W., Joty, S., & Hoi, S. C. (2021). CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In M.-F. Moens, X. Huang, L. Specia, & S. W.-t. Yih (Eds.), *Proceedings of the 2021 conference on empirical methods in natural language processing* (pp. 8696–8708). Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, <http://dx.doi.org/10.18653/v1/2021.emnlp-main.685>, URL <https://aclanthology.org/2021.emnlp-main.685>.
- Wang, X., Wei, J., Schuurmans, D., Le, Q. V., Chi, E. H., Narang, S., et al. (2023). Self-consistency improves chain of thought reasoning in language models. In *The eleventh international conference on learning representations, ICLR 2023, kigali, rwanda, May 1-5, 2023*. OpenReview.net, URL <https://openreview.net/pdf?id=1PL1NIMMrw>.
- Wang, L., Xu, W., Lan, Y., Hu, Z., Lan, Y., Lee, R. K.-W., et al. (2023). Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In A. Rogers, J. Boyd-Graber, N. Okazaki (Eds.), *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)* (pp. 2609–2634). Toronto, Canada: Association for Computational Linguistics, <http://dx.doi.org/10.18653/v1/2023.acl-long.147>, URL <https://aclanthology.org/2023.acl-long.147>.
- Wei, J., Bosma, M., Zhao, V., Guu, K., Yu, A. W., Lester, B., et al. (2022). Finetuned language models are zero-shot learners. In *International conference on learning representations*. URL <https://openreview.net/forum?id=gEzrGCozdqR>.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., et al. (2022). Chain of thought prompting elicits reasoning in large language models. In A. H. Oh, A. Agarwal, D. Belgrave, K. Cho (Eds.), *Advances in neural information processing systems*. URL <https://openreview.net/forum?id=VjQlMeSB.J>.
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., et al. (2020). Transformers: State-of-the-art natural language processing. In Q. Liu, D. Schlangen (Eds.), *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations* (pp. 38–45). Association for Computational Linguistics, <http://dx.doi.org/10.18653/v1/2020.emnlp-demos.6>, Online, URL <https://aclanthology.org/2020.emnlp-demos.6>.
- Yang, A., Xiao, B., Wang, B., Zhang, B., Bian, C., Yin, C., et al. (2023). Baichuan 2: Open large-scale language models. <http://dx.doi.org/10.48550/ARXIV.2309.10305>, CoRR arXiv:2309.10305.
- Yuan, Z., Yuan, H., Li, C., Dong, G., Tan, C., & Zhou, C. (2023). Scaling relationship on learning mathematical reasoning with large language models. <http://dx.doi.org/10.48550/ARXIV.2308.01825>, CoRR arXiv:2308.01825.
- Yue, X., Qu, X., Zhang, G., Fu, Y., Huang, W., Sun, H., et al. (2023). MAMmoTH: Building math generalist models through hybrid instruction tuning. <http://dx.doi.org/10.48550/ARXIV.2309.05653>, CoRR arXiv:2309.05653.
- Zhu, X., Li, J., Liu, Y., Ma, C., & Wang, W. (2023). A survey on model compression for large language models. <http://dx.doi.org/10.48550/ARXIV.2308.07633>, CoRR arXiv:2308.07633.
- Zhu, X., Qi, B., Zhang, K., Long, X., & Zhou, B. (2023). PaD: Program-aided distillation specializes large models in reasoning. <http://dx.doi.org/10.48550/ARXIV.2305.13888>, CoRR arXiv:2305.13888.